superpowers-bridge

Superpowers Bridge Reference

This document describes how superspec detects, invokes, and adapts obra/superpowers skills. The SKILL.md references this document when a command needs superpowers integration details.

Detection Logic

Superspec checks for superpowers skills at these paths, in order of precedence:

1. **Project-local**: `$PROJECT_DIR/.agents/skills/{skill-name}/SKILL.md`
2. **User-global**: `~/.agents/skills/{skill-name}/SKILL.md`

A skill is considered **available** if its `SKILL.md` file exists at either path. Project-local skills take precedence over user-global.

Detection Steps

When a superspec command needs a superpowers skill:

1. Determine the skill name from the mapping table below
2. Check project-local path first (use Glob or Read tool)
3. If not found, check user-global path
4. If found: log “Found {skill-name} at {path}” and proceed with enhanced mode
5. If not found: log “Superpowers {skill-name} not detected, using built-in fallback” and proceed with fallback mode

Skill Mapping

Superspec Command	Superpowers Skill Name	Skill Directory
/speckit.superspec.brainstorm	brainstorming	brainstorming/
/speckit.superspec.tasks	writing-plans	writing-plans/
/speckit.superspec.execute	executing-plans	executing-plans/
/speckit.superspec.execute	subagent-driven-development	subagent-driven-development/
/speckit.superspec.execute	test-driven-development	test-driven-development/
/speckit.superspec.review	requesting-code-review	requesting-code-review/

Invocation Pattern

When a superpowers skill is detected, the agent follows this pattern:

Step 1: Read the Skill

```
Read ~/.agents/skills/{skill-name}/SKILL.md
```

Parse the skill's instructions, process steps, and output expectations.

Step 2: Adapt Context

Before following the skill's process, establish context adaptation rules:

- **Input context**: Pass relevant spec-kit artifacts as context
  - Constitution: `.specify/memory/constitution.md`
  - Spec: `specs/NNN/spec.md`
  - Plan: `specs/NNN/plan.md` (if exists)
  - Tasks: `specs/NNN/tasks.md` (if exists)
- **Output location**: All outputs go to the `.specify/` structure
  - Superpowers may default to `docs/superpowers/` — redirect to `.specify/`
  - Brainstorming insights → update spec.md (Edge Cases, Open Questions, Brainstorm Log)
  - Writing-plans output → merge into `specs/NNN/tasks.md`

    - Code review findings → report to user, optionally write to checklist file
- **Naming conventions**: Use spec-kit conventions
    - Feature numbering: NNN (001, 002, …)
    - File naming: spec.md, plan.md, tasks.md (not design-doc.md, blueprint.md)

Step 3: Follow the Skill's Process

Execute the superpowers skill's steps as documented in its SKILL.md, but with the adapted context. The skill's methodology is the authority; only the input/output locations are adapted.

Step 4: Integrate Results

After the superpowers skill's process completes: - Verify outputs are in the correct `.specify/` locations - Update any cross-references (e.g., tasks.md referring to spec.md) - Log what was done in the relevant tracking section (e.g., Brainstorm Log)

Skill-Specific Adaptation Rules

brainstorming → `/speckit.superspec.brainstorm`

Process adaptation: - Follow the brainstorming skill's questioning protocol (one question at a time, Socratic method, challenge assumptions) - Apply questions to the target spec document - Record insights in the spec's "Open Questions" section and "Brainstorm Log" - The skill may produce a "design document" — fold its insights back into the existing spec.md rather than creating a separate file

Output mapping: | Superpowers Output | Superspec Destination | |———————–|———————–| | Design document | Update spec.md Edge Cases + Open Questions | | Clarified requirements | Update spec.md Functional Requirements | | Resolved questions | Update spec.md Open Questions table (mark Resolved) |

writing-plans → `/speckit.superspec.tasks`

Process adaptation: - Follow the writing-plans skill's task decomposition methodology - Structure the output using superspec's `tasks-template.md` format - Apply execution markers (`[TDD]`, `[REVIEW]`, `[SUBAGENT]`, `[P]`) based on the plan's execution strategy section

Output mapping: | Superpowers Output | Superspec Destination | |—————————–|—————————–| | Implementation blueprint | `specs/NNN/tasks.md` | | Task dependencies | Tasks Dependencies section | | Parallel opportunities | Tasks marked with `[P]` and `[SUBAGENT]` |

executing-plans → `/speckit.superspec.execute`

Process adaptation: - Follow the executing-plans skill's batch processing protocol - Respect checkpoint gates defined in tasks.md - Apply human checkpoint protocol at every phase boundary - Never auto-approve — always wait for user confirmation

Combined with: - `subagent-driven-development`: For tasks marked `[SUBAGENT]`, follow this skill's dispatch protocol to delegate work to subagents - `test-driven-development`: For tasks marked `[TDD]`, follow this skill's RED-GREEN-REFACTOR discipline

requesting-code-review → `/speckit.superspec.review`

Process adaptation: - Follow the requesting-code-review skill's pre-evaluation checklist - Add superspec-specific review dimensions: - Spec compliance (acceptance scenarios from spec.md) - Constitution compliance (principles from constitution.md) - Brainstorm coverage (edge cases from brainstorming sessions) - Report findings with confidence scores (0-100, threshold >= 80)

Output mapping: | Superpowers Output | Superspec Destination | |—————————–|—————————–| | Review findings | Reported to user | | Checklist | Optional: `specs/NNN/checklist-review.md` |

Graceful Degradation

Every superpowers integration has a built-in fallback. The skill NEVER hard-fails when superpowers are not installed.

Fallback Behavior Summary

Superpowers Skill	Fallback	Where Defined
`brainstorming`	5-category questioning protocol (boundary, error, scale, security, UX)	workflow-guide.md Phase 2
`writing-plans`	Direct template-based	workflow-guide.md Phase 4

Superpowers Skill	Fallback	Where Defined
	decomposition from plan	
`executing-plans`	Sequential task walk with manual confirmation at checkpoints	workflow-guide.md Phase 5
`subagent-driven-development`	Sequential in-session implementation (no parallel dispatch)	workflow-guide.md Phase 5
`test-driven-development`	Inline TDD: write test → verify fail → implement → verify pass	workflow-guide.md Phase 5
`requesting-code-review`	Built-in review checklist (spec compliance, constitution, code quality)	workflow-guide.md Phase 6

Fallback Quality

The built-in fallbacks are designed to be **functional but lighter-weight** than the full superpowers experience:

- **Brainstorming fallback**: Covers the same 5 categories but with simpler questioning patterns. Superpowers brainstorming adds Socratic method, assumption challenging, and more nuanced exploration.
- **Writing-plans fallback**: Produces a solid task breakdown from the plan template. Superpowers writing-plans adds more sophisticated dependency analysis and parallelization identification.
- **Execution fallback**: Walks tasks sequentially with manual checkpoints. Superpowers adds intelligent batching, parallel subagent dispatch, and automated checkpoint evaluation.
- **Review fallback**: Uses a static checklist approach. Superpowers adds multi-agent parallel review with confidence scoring and false-positive filtering.

Troubleshooting

Skills Not Detected

If superpowers skills are installed but not detected:

1. Verify the skill directory name matches exactly (case-sensitive)
2. Verify the SKILL.md file exists inside the skill directory
3. Check both paths: `.agents/skills/` (project) and `~/.agents/skills/` (global)
4. Verify file permissions allow reading

Skill Process Conflicts

If the superpowers skill's process conflicts with superspec conventions:

1. Superspec output locations always take precedence (write to `.specify/`)
2. Superspec naming conventions always take precedence (spec.md, plan.md, tasks.md)
3. The skill's methodology and questioning approach take precedence over fallback
4. When in doubt: follow the superpowers process, adapt only the outputs